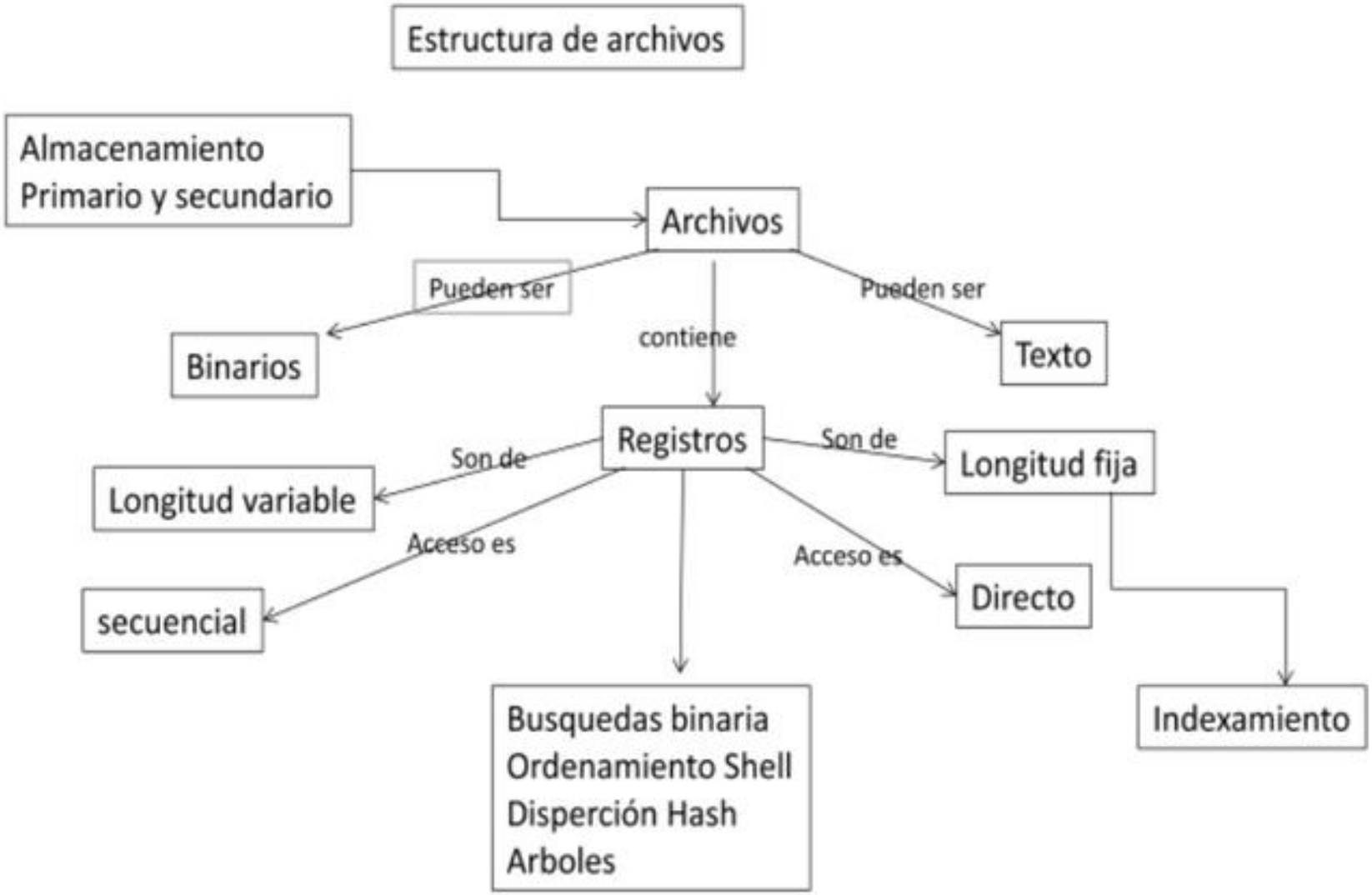
Motores de Búsqueda



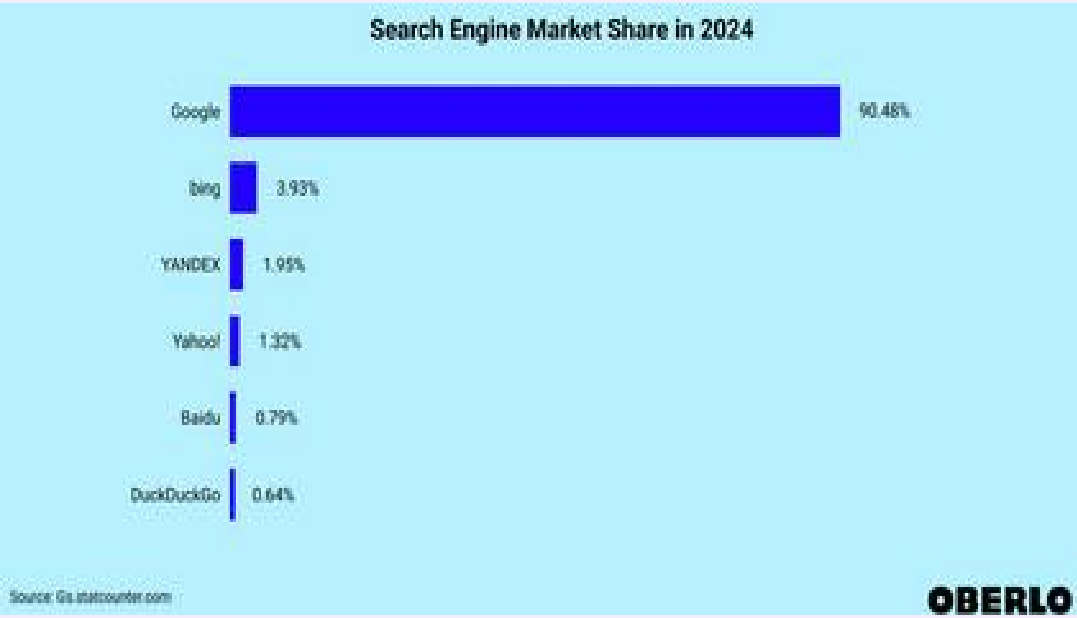
Motores de Búsqueda

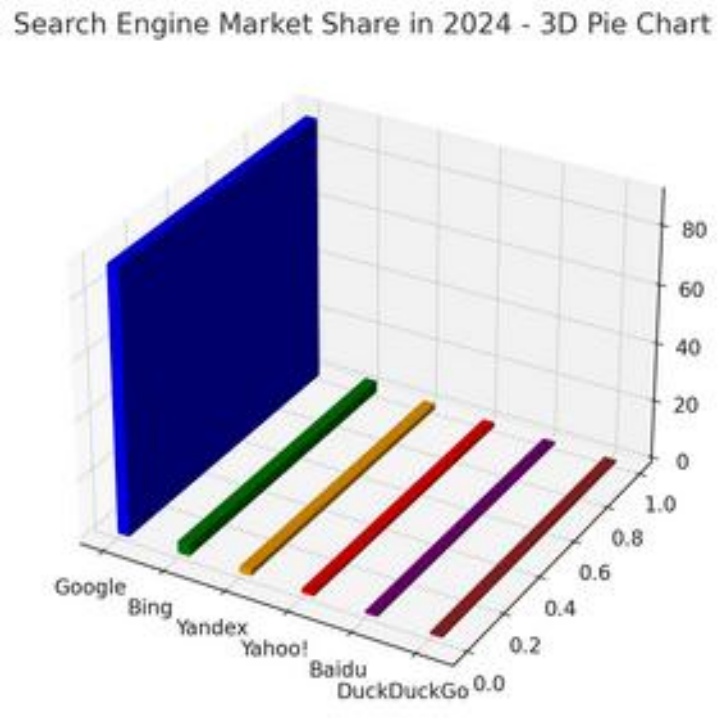
Motores de Búsqueda

Que es un Motor de Búsqueda:

Un sistema que indexa y responde consultas rápidamente

Motores de Búsqueda





Motores de Búsqueda

Motores de Búsqueda

Definición:

Un sistema que:

- ***Construye un índice a partir de texto***
- ***Responde a consultas utilizando este índice***

Un motor de búsquedas ofrece:

- ***Escalabilidad***
- ***Ranking de relevancia***
- ***Integra diferentes fuentes de datos*** *(mail, pag. web, archivos, etc.)*

Trabaja con Archivos invertidos

- ***Utiliza índices invertidos.***

Motores de Búsqueda

Motores de Búsqueda

¿Como funciona?

Proceso de Indexación:

- Proceso de indexación:
- Convertir el documento
- Extraer texto y metadatos
- Normalizar el texto
- Crear el índice invertido.

- Como lo hace:
  - Evaluando Palabras, no sub-strings

    ej. auto <> automóvil, automático,...

Motores de Búsqueda

¿Como funciona?

Ejemplo de funcionamiento:

- ***Tenemos dos documentos:***
  - ***Archivo 1:***
    "El motor de búsqueda utiliza índices invertidos de strings".
  - ***Archivo 2:***
    "Los Strings de los índices a son esenciales para el motor"
- ***Índice Invertido:***

Motor	***1,2***	***índices***	***1,2***
Strings	***1,2***	***utiliza***	***1***
Búsqueda	***1***	***invertidos***	***1***
Invertidos	***1***	***esenciales***	***2***

Motores de Búsqueda

Estructura basada en índices invertidos

Archivo Invertido:

- *La estructura primaria de se basa en archivos invertidos.*
- *Consiste en utilizar palabras para indexar una colección de texto con el objetivo de acelerar la tarea de búsqueda*

Estructura archivo inverso:

- ***Diccionario:***
  - ***el conjunto de todas las palabras distintas que aparecen en el texto***
- ***Ocurrencias (postings):***
  - ***listas conteniendo toda la información necesaria para cada palabra del vocabulario (documentos donde aparece, posición, frecuencia, etc.)***

Motores de Búsqueda

Estructura basada en índices invertidos

Archivo Invertido:

- *Palabras indexadas por su aparición en el texto, acelerando las búsquedas.*
- *Un archivo invertido es esencialmente un vector invertido tal que las columnas se vuelven filas y las filas columnas:*

Docs	t1	t2	t3
D1	1	0	1
D2	1	0	0
D3	0	1	1
D4	1	0	0
D5	1	1	1
D6	1	1	0
D7	0	1	0
D8	0	1	0
D9	0	0	1
D10	0	1	1
D11	1	0	1

Términos	D1	D2	D3	D4	D5	D6	D7	...
t1	1	1	0	1	1	1	0	
t2	0	0	1	0	1	1	1	
t3	1	0	1	0	1	0	0	

Motores de Búsqueda

Pasos para Construir un Motor de Búsqueda

<u>El proceso involucra cuatro pasos claves:</u>

- *1) Parseo*
- *2) Ordenamiento*
- *3) Agrupamiento.*
- *4) Generación del Diccionario y Posting.*

Motores de Búsqueda

Pasos para la construcción

1) Parseo:

A cada Documento extraerle los términos y Id Documento

Documento 1

Un fanático es alguien que no puede cambiar de opinión y no quiere cambiar de tema

Documento 2

El fanático suele manifestarse con una pasión exagerada, desmedida cuando expone el tema que lo fanatiza.

Termino	D#
Un	1
fanático	1
es	1
Alguien	1
que	1
no	1
puede	1
cambiar	1
de	1
opinión	1
Y	1
no	1
quiere	1
cambiar	1
De	1
tema	1

Termino	D#
El	2
fanático	2
suele	2
manifestarse	2
con	2
Una	2
pasión	2
exagerada	2
desmedida	2
cuando	2
expone	2
el	2
tema	2
que	2
lo	2
fanatiza	2

Motores de Búsqueda

Pasos para la construcción

2) Ordenamiento:

Ordenar al archivo por Términos.
Se mantienen los duplicados.

Termino	D#
Un	1
fanático	1
es	1
Alguien	1
que	1
no	1
puede	1
cambiar	1
de	1
opinión	1
Y	1
no	1
quiere	1
cambiar	1
De	1
tema	1

Termino	D#
El	2
fanático	2
suele	2
manifestarse	2
con	2
Una	2
pasión	2
exagerada	2
desmedida	2
cuando	2
expone	2
el	2
tema	2
que	2
lo	2
fanatiza	2

Ordenar

Termino	D#
aguien	1
cambiar	1
cambiar	1
con	2
cuando	2
de	1
de	1
desmedida	2
el	2
el	2
es	1
exagerada	2
expone	2
fanático	1
fanático	2
fanático	2

Termino	D#
lo	2
manifestarse	2
no	1
no	1
opinión	1
pasión	2
puede	1
que	1
que	2
quiere	1
suele	2
tema	1
tema	2
un	1
una	2
y	1

Motores de Búsqueda

Pasos para la construcción

3) Agrupamiento:

Corte de control por termino y Documento.

Termino	D#
aguien	1
cambiar	1
cambiar	1
con	2
cuando	2
de	1
de	1
desmedida	2
el	2
el	2
es	1
exagerada	2
expone	2
fanático	1
fanático	2
fanático	2

Termino	D#
lo	2
manifestarse	2
no	1
no	1
opinión	1
pasión	2
puede	1
que	1
que	2
quiere	1
suele	2
tema	1
tema	2
un	1
una	2
y	1

Agrupar →

Termino	D#	F#
alguien	1	1
cambiar	1	2
con	2	1
cuando	2	1
de	1	2
desmedida	2	1
el	2	2
es	1	1
exagerada	2	1
expone	2	1
fanático	1	1
fanático	2	1
fanatiza	2	1
lo	2	1

Termino	D#	F#
manifestarse	2	1
no	1	2
opinión	1	1
pasión	2	1
puede	1	1
que	1	1
que	2	1
quiere	1	1
suele	2	1
tema	1	1
tema	2	1
un	1	1
una	2	1
y	1	1

Motores de Búsqueda

Pasos para la construcción

4) Diccionario y Posting:

Generación del Diccionario
Generación archivo posting

Termino	D#	F#
alguien	1	1
cambiar	1	2
con	2	1
cuando	2	1
de	1	2
desmedida	2	1
el	2	2
es	1	1
exagerada	2	1
expone	2	1
fanático	1	1
fanático	2	1
fanatiza	2	1
lo	2	1

Termino	D#	F#
manifestarse	2	1
no	1	2
opinión	1	1
pasión	2	1
puede	1	1
que	1	1
que	2	1
quiere	1	1
suele	2	1
tema	1	1
tema	2	1
un	1	1
una	2	1
y	1	1

Generar

Diccionario
Índices Invertido

Posting
Índices Lexicográfico, Hashing, B+, etc

Datos
Ubicación, url, etc

Motores de Búsqueda

Pasos para la construcción

4) Diccionario y Posting:

Diccionario:

- Agrupa por termino.
- Almacena Frecuencia. Tot.
- Direcciona al Posting.

Archivo inverso con Índice no denso.

Posting:

- Direcciona (ubicación de los documentos, url, etc.).
- Almacena Frecuencia de cada documento.

Archivo con lista inversa.

Termino	TD#	FT#	#RR
alguien	1	1	1
cambiar	1	2	2
con	1	1	3
cuando	1	1	4
de	1	2	5
desmedida	1	1	6
el	1	2	7
es	1	1	8
exagerada	1	1	9
expone	1	1	10
fanático	2	2	11
fanatiza	1	1	13
lo	1	1	14
manifestarse	1	1	15
no	1	2	16
opinión	1	1	17
pasión	1	1	18
puede	1	1	19
que	2	2	20
quiere	1	1	22
suele	1	1	23
tema	2	2	24
un	1	1	26
una	1	1	27
y	1	1	28

#RR	D#	F#	#PR
1	1	1	-1
2	1	2	-1
3	2	1	-1
4	2	1	-1
5	1	2	-1
6	2	1	-1
7	2	2	-1
8	1	1	-1
9	2	1	-1
10	2	1	-1
11	1	1	12
12	2	1	-1
13	2	1	-1
14	2	1	-1
15	2	1	-1
16	1	2	-1
17	1	1	-1
18	2	1	-1
19	1	1	-1
20	1	1	21
21	2	1	-1
22	1	1	-1
23	2	1	-1
24	1	1	25
25	2	1	-1
26	1	1	-1
27	2	1	-1
28	1	1	-1

Motores de Búsqueda

Ejemplo Vectorial

D1: "Un motor de búsqueda utiliza indices invertidos"

D2: "El motor de Google utiliza índices"

D3: "Los Indices del motor de búsqueda son indices invertidos"

Su ubicación:

D1: /home/docs/d1.txt.

D2: https://sitio/carpeta/d2.txt.

D3: /carpeta/documentos/d3.txt.

Vamos a resolverlo paso a paso.

Motores de Búsqueda

Paso1: Eliminación de los StopWords

Extraer los términos relevantes.

Las **stopwords** son palabras comunes que los motores de búsqueda ignoran para mejorar la relevancia y eficiencia en la búsqueda de información.

D1: "motor búsqueda utiliza índices invertidos"

D2: "motor Google utiliza índices"

D3: "índices, motor, búsqueda, invertidos"

Motores de Búsqueda

Paso 2: Construir el Indice Ivertido

Frecuencia de los términos en cada documento:

Término	D1	D2	D3
motor	1	1	1
búsqueda	1	0	1
utiliza	1	1	0
índices	1	1	2
invertidos	1	0	1
google	0	1	0

Motores de Búsqueda

Paso 2: Construir el Indice Ivertido

Índice Invertido

Término	Documentos en los que aparece
motor	D1, D2, D3
búsqueda	D1, D3
utiliza	D1, D2
índices	D1, D2, D3
invertidos	D1, D3
google	D2

Motores de Búsqueda

Paso 3: Diccionario de Terminos y Frecuencia:

Término	TD# (Documentos)	FT# (Frecuencia total)	#RR
motor	3	3	1
búsqueda	2	2	2
utiliza	2	2	3
índices	3	4	4
invertidos	2	2	5
google	1	1	6

Motores de Búsqueda

Paso 3: Diccionario de Terminos y Frecuencia:

#RR	D# (Documento)	F# (Frecuencia en Documento)	PR (Posición en Documento)
1	D1: /home/docs/d1.txt	1	2
1	D2: https://sitio/carpeta/d2.txt	1	2
1	D3: /carpeta/documentos/ d3.txt	1	4
2	D1: /home/docs/d1.txt	1	4
2	D3: /carpeta/documentos/ d3.txt	1	5
3	D1: /home/docs/d1.txt	1	5
3	D2: https://sitio/carpeta/d2.txt	1	5
4	D1: /home/docs/d1.txt	1	6
4	D2: https://sitio/carpeta/d2.txt	1	6
4	D3: /carpeta/documentos/ d3.txt	2	2, 6
5	D1: /home/docs/d1.txt	1	7
5	D3: /carpeta/documentos/ d3.txt	1	7
6	D2: https://sitio/carpeta/d2.txt	1	4

Motores de Búsqueda

Consultando por “Indices”:

Paso 1 Buscamos el término "**índices**" en la tabla de Termnos y frecuancia. Aparece en (TD#) **3** documentos con una frecuencia total de (FT#) **4**. Su posición en el Round Robin (#**RR) es 4.**

Término	TD# (Documentos)	FT# (Frecuencia total)	#RR
índices	3	4	4

Paso 2 Encontramos el recorrido de "**índices**" en la **Posting** , donde #**RR = 4:**

#RR	D# (Documento)	F# (Frecuencia en Documento)	PR (Posición en Documento)
4	D1: /home/docs/d1.txt	1	6
4	D2: https://sitio/carpeta/d2.txt	1	6
4	D3: /carpeta/documentos/ d3.txt	2	2, 6

Motores de Búsqueda

Consultando por “Indices”:

Resultado del Recorrido

1. **Documento 1**:
   - Ruta: `/home/docs/d1.txt`
   - Frecuencia: 1 aparición
   - Posición: 6
2. **Documento 2**:
   - Ruta: `https://sitio/carpeta/d2.txt`
   - Frecuencia: 1 aparición
   - Posición: 6
3. **Documento 3**:
   - Ruta: `/carpeta/documentos/d3.txt`
   - Frecuencia: 2 apariciones
   - Posiciones: 2, 6

Motores de Búsqueda

Búsquedas sobre el archi

La búsqueda más simple es utilizando una única palabra
Se realiza utilizando una estructura de datos adecuada
(por ej. hashtables, trees o b-trees)

Las consultas conjuntivas implican:

Buscar todas las palabras de la consulta
Es la forma más popular en la Web
Requiere la intersección de todas las listas invertidas

Las consultas disyuntivas implican:

La unión de las listas

Eje. De Consultas:

fanático And tema.
fanático or tema

Motores de Búsqueda

Búsquedas archivo invertido

Las consultas contextuales son más difíciles de resolver con índices invertidos:

Todas las palabras aparecen en secuencia (una frase). aparecen lo suficientemente cerca (por proximidad)las soluciones incluyen indexación de pares de palabras o índices posicionales, pero se incrementa el tamaño del índice.

Búsqueda por prefijo o por rango son básicamente consultas disyuntivas mayores.

Para las búsquedas de expresiones regulares no sirven las estructuras de datos construidas sobre el vocabulario, requieren una búsqueda secuencial

Recuperación de Textos (RTT)

Recuperación de Textos

Distintas variantes de Búsqueda:

Operadores:

Conjunción: (and , +):
Los documentos recuperados deben tener todos los términos de la búsqueda

Disyunción (or , -):
Las Unidades de recuperación (los documentos), deben contener Alguno de los términos específicos

Exclusión (not):
Los documentos no deben tener los términos especificados.

Recuperación de Textos (RTT)

Recuperación de Textos

Características de los Modelos:

Modelo Booleano:

- Es el primer Modelo implementado y el mas usado.
- Simple de implementar.
- Las implementaciones son eficientes.
- Es el mas utilizado en administración de Bases de Datos.
- Los resultados de las búsquedas tienen todas la misma
- Relevancia.
- No contempla el cumplimiento parcial.
- No contempla la cantidad de veces que una palabra se encuentra.
- Muchos usuarios no entienden las reglas lógicas.

Recuperación de Textos (RTT)

Recuperación de Textos

Características de los Modelos:

Modelo Vectorial:

- **Dado un conjunto de documentos (D):**

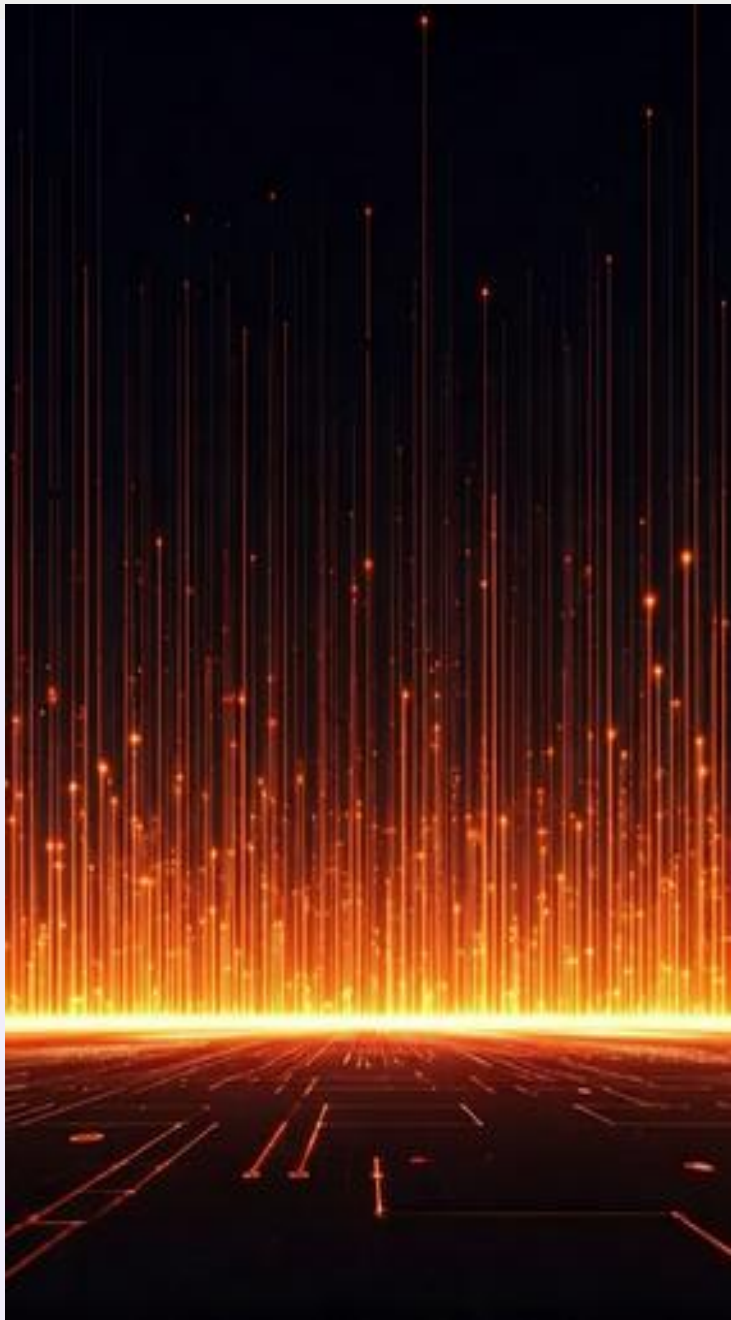
  D= { d1, d2, d3,..,dn }

- Un conjunto de todos los términos relevantes del Conjunto D.
- T= { t1, t2, t3,..tn }

Se modela vectorial mente:

Las Unidades de recuperación (los documentos), deben contener Alguno de los términos específicos

O bien en la exclusión (not):
Los documentos no deben tener los términos especificados.



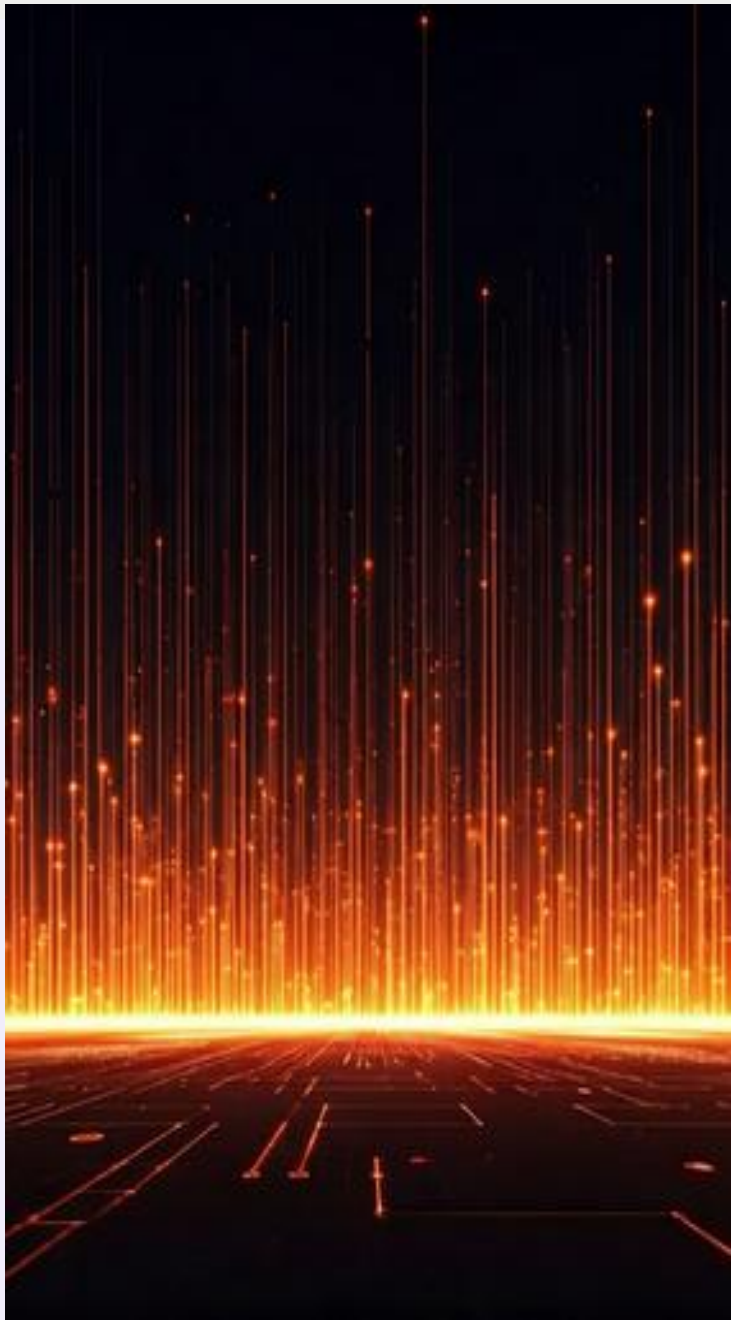
Recuperación de Textos (RTT)

Recuperación de Textos

Características de los Modelos:

Modelo Vectorial:

- El termino tiene un peso en el documento.
- Cuanto mayor frecuencia, mayor peso tiene.
- **Ley de Zipf:** Puede haber alta frecuencia de términos globales Y baja frecuencia en cada documento.
- Los vectores se normalizan para neutralizar los tamaños de los documentos con los términos
- Permite ordenar documentos por su relevancia.
- Permite agrupar documentos similares
- **SIMILITUD:** 2 Documentos iguales Similitud=1.
  2 Documentos distintos Similitud=0



Recuperación de Textos (RTT)

Recuperación de Textos

Características de los Modelos:

Ley de Zipf:

- Describe la distribución de frecuencias de ciertos tipos de datos en una variedad de fenómenos naturales y lenguajes. El termino tiene un peso en el documento.

- Establece que en un conjunto de datos, como palabras en un texto, la frecuencia de un elemento es inversamente proporcional a su rango en la lista ordenada de frecuencias.

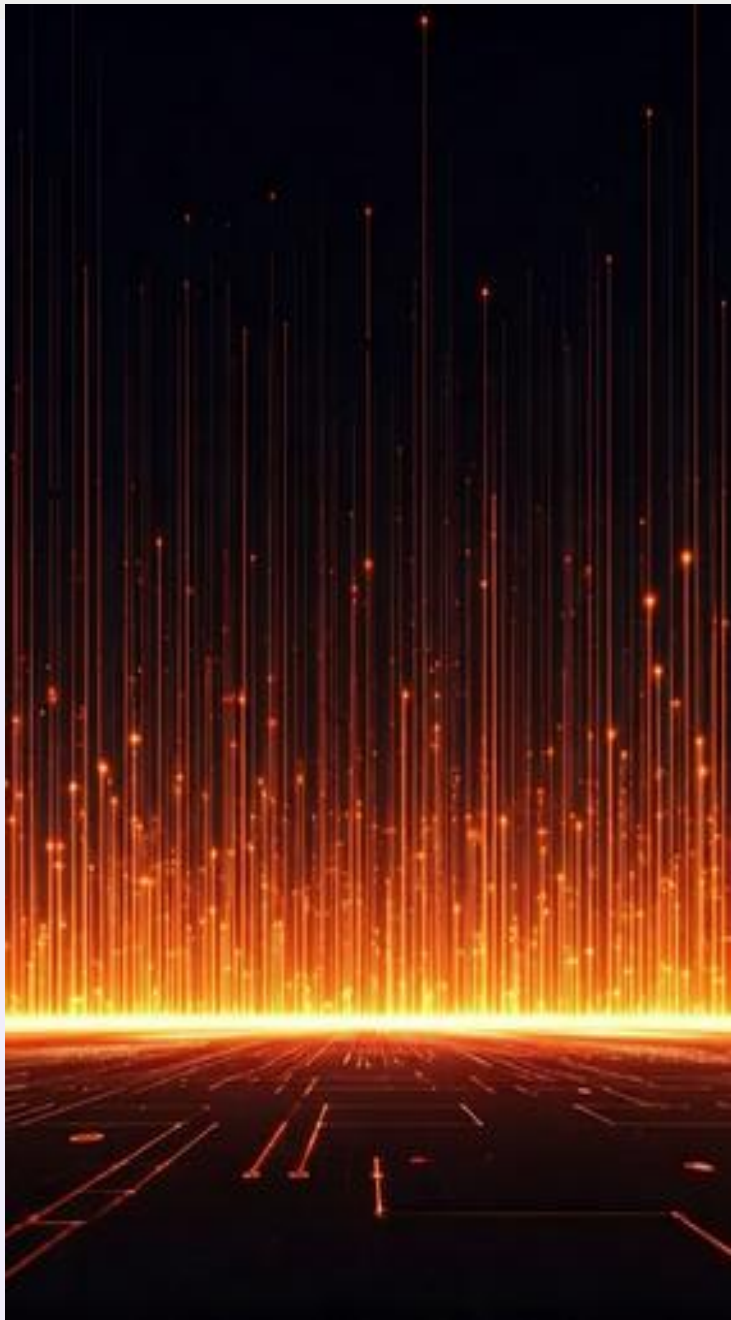
  Ejemplo: Contar palabras de un libro

  La palabra más frecuente podría ser "el" (muy común en español), que aparece, por ejemplo, 1000 veces.

  La segunda palabra más frecuente ("de", por ejemplo) aparecería aproximadamente 500 veces.

  La tercera palabra más frecuente ("y") podría aparecer unas 333 veces.

- **Aplicaciones:** Lenguaje Natural – Distribución de Ingresos - Analisis de Web.



Motores de Búsqueda

Consultas especiales

Frases Textuales:

Títulos de obras, procesos o hechos históricos, etc.

(p.e. “Don segundo Sombra”,”El modelo TCP”, “Platero y yo”), nombres compuestos (p.e. “San José de Costa Rica””).

No basta si los términos están en un documento. Hay que saber además sus posiciones relativas en mismo.

Términos Próximos:

Son para buscar frases equivalentes.

(p.ep. “Modelos e implementación” o “Implementación y Modelos”)

Se puede pedir que los Términos estén a menos de una distancia.

Hay que conocer la ubicación relativa de los términos en el documento. (numero de Bloque o párrafo donde aparece)

Motores de Búsqueda

Consultas especiales

Patrones:

- Utilización de comodines (* y ?):
  Tipos de caracteres repetibles u opcionales
  (p.e. “capitulo [0-9].doc?”)
- Expresiones regulares
  (p.e. “(Sr.|Sra.) Martinez”)

Términos parecidos:

- No estoy seguro de como se escribe.
  Tipico en palabras mal escritas , OCR, etc.
  (p.ep. ~“himmen”)

Motores de Búsqueda

Índices Invertidos

Como vimos:

Utilizamos archivos invertidos para resolver El diccionario y el posting.

- Se basan en la clasificación por termino.
- El conjunto de términos de un documento se los llama: Vocabulario o léxico.
- Las listas de términos se las llama invertidas.

Modelo Booleano:

- Se organizan en orden creciente de documento para optimizar operaciones de conjunto.

Modelo Vectorial:

- Cada elemento de a lista invertida de términos es para cada documento que aparezca.

Para búsqueda de Frases y términos:

- Para cada termino se debe tener la lista de posiciones donde aparece en el documento.

Motores de Búsqueda

Índices Invertidos

d1 Las Cosas de la Vida

d2 La Vida es Bella

d3 Las Cosas del Querer

d4 La Vida después de la Vida

MODELO BOOLEANO

Vocabulario	Listas Invertidas
bella	2
cosas	1, 3
querer	3
vida	1, 2, 4

Motores de Búsqueda

Índices Invertidos

d1 Las Cosas de la Vida

d2 La Vida es Bella

d3 Las Cosas del Querer

d4 La Vida después de la Vida

MODELO VECTORIAL

Vocabulario	Listas Invertidas
bella	(2, 1)
cosas	(1, 1), (3, 1)
querer	(3, 1)
vida	(1,1), (2, 1), (4, 2)

Motores de Búsqueda

Índices Invertidos

d1 Las Cosas de la Vida

d2 La Vida es Bella

d3 Las Cosas del Querer

d4 La Vida después de la Vida

FRASES Y TÉRMINOS PRÓXIMOS

Vocabulario	Listas Invertidas
bella	2 (2)
cosas	1 (1), 3 (1)
querer	3 (2)
vida	1 (2), 2 (1), 4 (1, 2)

Recuperación de Textos (RTT)

Recuperación de Textos

- ***Índices Invertidos***

d1 Las Cosas de la Vida

d2 La Vida es Bella

d3 Las Cosas del Querer

d4 La Vida después de la Vida

MODELO BOOLEANO

Vocabulario	Listas Invertidas
bella	2
cosas	1, 3
querer	3
vida	1, 2, 4

MODELO VECTORIAL

Vocabulario	Listas Invertidas
bella	(2, 1)
cosas	(1, 1), (3, 1)
querer	(3, 1)
vida	(1,1), (2, 1), (4, 2)

FRASES Y TÉRMINOS PRÓXIMOS

Vocabulario	Listas Invertidas
bella	2 (2)
cosas	1 (1), 3 (1)
querer	3 (2)
vida	1 (2), 2 (1), 4 (1, 2)

Motores de Búsqueda

Índices de firmas

La firma de un documento sirve ara verificar si un termino esta contenido.

Para un tamaño X de bits de la firma. Se aplica hashing

- Entre 0 y X-1. A los términos
- Se almacenan en un archivo secuencial con registros Correspondientes a la firma de cada documento.
- Solo responde al modelo Booleano.
- Bitslice signature file, es el archivo donde se guardan las Firmas, que representan los términos del documento.
- La búsqueda en el archivo de Firmas es secuencial.

Motores de Búsqueda

Ejemplo indices de firmas

D1: "Un motor de búsqueda utiliza indices invertidos"

D2: "El motor de Google utiliza índices"

D3: "Los Indices del motor de búsqueda son indices invertidos"

Su ubicación:

D1: /home/docs/d1.txt.

D2: https://sitio/carpeta/d2.txt.

D3: /carpeta/documentos/d3.txt.

Vamos a resolverlo paso a paso.

Motores de Búsqueda

Paso1: Eliminación de los StopWords

Extraer los términos relevantes.

Las **stopwords** son palabras comunes que los motores de búsqueda ignoran para mejorar la relevancia y eficiencia en la búsqueda de información.

D1: "motor búsqueda utiliza índices invertidos"

D2: "motor Google utiliza índices"

D3: "índices, motor, búsqueda, invertidos"

Motores de Búsqueda

Paso 2: Términos del Vocabulario

Extraer los términos relevantes.

Términos Del Vocabulario

	Número	Términos
1	1	motor
2	2	búsqueda
3	3	utiliza
4	4	índices
5	5	google

Motores de Búsqueda

Paso 3: Asignar bits a los términos

Cada término tiene un bit asignado:

Bit	Término
1	motor
2	búsqueda
3	utiliza
4	índices
5	invertidos
6	google

Paso 4: Generación de Firmas

D1:("motor,búsqueda,utiliza,índices,invertidos"): **111110**

D2:("motor,Google,utiliza,índices"): **101110**

D3:("índices,motor,búsqueda,índices,invertidos"): **111110**

BitMaps

Motores de Búsqueda

Paso 5: Bitslice Signature File

Asocia el estado de cada bit con cada documento

Bit	D1	D2	D3
1	1	1	1
2	1	0	1
3	1	1	0
4	1	1	1
5	1	0	1
6	0	1	0

Motores de Búsqueda

consulta1: Bitslice Signature File

Buscamos “motor” e “indices”

Términos de la búsqueda:

- "motor" → Bit 1
- "índices" → Bit 4

Paso 1: Revisamos el bit 1 para todos los documentos

Bit 1 (motor)	D1	D2	D3
	1	1	1

Paso 2: Revisamos el bit 4 para todos los documentos

Bit 4 (índices)	D1	D2	D3
	1	1	1

Los documentos **D1**, **D2**, y **D3** contienen ambos términos ("motor" e "índices").

Ubicaciones:

- D1: `/home/docs/d1.txt`
- D2: `https://sitio/carpeta/d2.txt`
- D3: `/carpeta/documentos/d3.txt`

Bit	Término
1	motor
2	búsqueda
3	utiliza
4	índices
5	invertidos
6	google

Bit	Término
1	motor
2	búsqueda
3	utiliza
4	índices
5	invertidos
6	google

Motores de Búsqueda

consulta2: Bitslice Signature File

Buscamos “google” e “utiliza”

Términos de la búsqueda:

- "google" → Bit 6
- "utiliza" → Bit 3

Paso 1: Revisamos el bit 6 para todos los documentos

Bit 6 (google)	D1	D2	D3
	0	1	0

Paso 2: Revisamos el bit 3 para todos los documentos

Bit 3 (utiliza)	D1	D2	D3
	1	1	0

Solo el documento **D2** contiene ambos términos ("google" y "utiliza").

Ubicación:

- D2: `https://sitio/carpeta/d2.txt`

Bit	Término
1	motor
2	búsqueda
3	utiliza
4	índices
5	invertidos
6	google

Bit	Término
1	motor
2	búsqueda
3	utiliza
4	índices
5	invertidos
6	google

Motores de Búsqueda

Ejemplo indices de firmas con HASH

D1: "Un motor de búsqueda utiliza indices invertidos"

D2: "El motor de Google utiliza índices"

D3: "Los Indices del motor de búsqueda son indices invertidos"

Su ubicación:

D1: /home/docs/d1.txt.

D2: https://sitio/carpeta/d2.txt.

D3: /carpeta/documentos/d3.txt.

Vamos a resolverlo paso a paso.

Motores de Búsqueda

Paso 2: Términos del Vocabulario

Salimos del ejemplo anterior los términos relevantes.

Términos Del Vocabulario

	Número	Términos
1	1	motor
2	2	búsqueda
3	3	utiliza
4	4	índices
5	5	google

Motores de Búsqueda

Paso 3: Hashing de los Términos

Salimos del ejemplo anterior los términos relevantes.

Término	Hash (mod 8)	Bit activado
motor	hash("motor") % 8 = 1	Bit 1
búsqueda	hash("búsqueda") % 8 = 3	Bit 3
utiliza	hash("utiliza") % 8 = 5	Bit 5
indices	hash("indices") % 8 = 2	Bit 2
invertidos	hash("invertidos") % 8 = 7	Bit 7
google	hash("google") % 8 = 4	Bit 4

Paso-3:Bitslice-Signature-File-hashing:

Bit	D1	D2	D3
1	1	1	1
2	1	1	1
3	1	0	1
4	0	1	0
5	1	1	0
6	0	0	0
7	1	0	1
8	0	0	0

Motores de Búsqueda

Paso 4: Hashing de documentos

Así se vería la tabla Hash para acceso a los documentos

Índice (Hash)	Término	Documentos
0	(vacío)	-
1	motor	D1: `/home/docs/d1.txt`, D2: `https://sitio/carpeta/d2.txt`, D3: `/carpeta/documentos/d3.txt`
2	índices	D1: `/home/docs/d1.txt`, D2: `https://sitio/carpeta/d2.txt`, D3: `/carpeta/documentos/d3.txt`
3	búsqueda	D1: `/home/docs/d1.txt`, D3: `/carpeta/documentos/d3.txt`
4	google	D2: `https://sitio/carpeta/d2.txt`
5	utiliza	D1: `/home/docs/d1.txt`, D2: `https://sitio/carpeta/d2.txt`
6	(vacío)	-
7	invertidos	D1: `/home/docs/d1.txt`, D3: `/carpeta/documentos/d3.txt`

La consulta requiere calcular el Hash del termino

Motores de Búsqueda

Consulta con Patrones

Fuerza bruta: Recorrido secuencial del léxico

Índice secundario de n-gramas: Por cada término que se agrega al índice invertido, se indexan en el secundario todos sus n-gramas

Índice secundario de léxico rotado: Por cada término que se agrega al índice invertido, se agregan al secundario todas sus rotaciones

Los índices secundarios permiten buscar en el invertido todos los términos que cumplan con el patrón de búsqueda

Motores de Búsqueda

Consulta con Patrones

Índice Secundario de n-gramas:

Para agregar el término normalizado "organizacion" al índice invertido y teniendo un índice secundario de digrama, deben indexarse los digramas:

Índice Secundario De Digrama (2) De 'Organizacion'

	Digrama	Término
1	or	organizacion
2	rg	organizacion
3	ga	organizacion
4	an	organizacion
5	ni	organizacion
6	iz	organizacion

Búsqueda Del Patrón 'Niz'

		Digrama	Términos asociados
1	0	ni	['organizacion']
2	1	iz	['organizacion']
3	2	Intersección	['organizacion']

Para buscar un patrón orga* se buscan las listas de términos correspondientes a los digramas o,or,rg,ga y se hace la intersección de las listas.

Motores de Búsqueda

Consulta con Patrones

Índice Secundario de Léxico Rotado:

Para agregar el término normalizado "organizacion" al índice invertido deben indexarse las rotaciones:

Rotaciones De 'Organizacion'

	Rotación	Término
1	organizacion	organizacion
2	nizaciono	organizacion
3	izacionon	organizacion
4	zaciononi	organizacion
5	aciononiz	organizacion
6	ciononiza	organizacion

Búsqueda Del Patrón 'Orga'

		Rotación	Coincide con "orga"
1	0	organizacion	True

Búsqueda Del Patrón 'Oni'

		Rotación	Coincide con "oni"
1	3	zaciononi	True
2	4	aciononiz	True
3	5	ciononiza	True
4	6	iononiza c	True
5	7	ononizaci	True
6	8	nonizacio	True

Para buscar un patrón orga* se buscan los términos correspondientes a todas las rotaciones que comiencen con orgaorga.

Motores de Búsqueda

Espacio Métrico (Léxico, Distancia Edición)

Distancia de Edición de Levenshtein: Número mínimo de operaciones requeridas para transformar una cadena de caracteres en otra.

Inserción: Agregar un carácter a la cadena.
Eliminación: Quitar un carácter de la cadena.
Sustitución: Cambiar un carácter por otro.

Ejemplo 1:

Para las cadenas **"gato"** y **"pato"**:

- $A = \text{"gato"}$
- $B = \text{"pato"}$

Podemos cambiar la 'g' por 'p':

- Sustitución: **g**ato → **p**ato.

Distancia de Levenshtein = 1.

Ejemplo 2:

Para las cadenas **"casa"** y **"caza"**:

- $A = \text{"casa"}$
- $B = \text{"caza"}$

Podemos cambiar la 's' por 'z':

- Sustitución: ca**s**a → ca**z**a.

Distancia de Levenshtein = 1.

Ejemplo 3:

Para las cadenas **"silla"** y **"salir"**:

- $A = \text{"silla"}$
- $B = \text{"salir"}$

Podemos hacer los siguientes cambios:

1. Cambiar 'i' por 'a': s**i**lla → s**a**lla.
2. Cambiar la 'l' por 'l': no requiere cambio.
3. Cambiar 'a' por 'r': sa**l**la → sa**l**ir.

Distancia de Levenshtein = 3 (dos sustituciones y una eliminación).

Es una generalización de la distancia de Hamming (Aplicada por bits) para códigos autocorrectores.

Motores de Búsqueda

Espacio Métrico (Léxico, Distancia Edición

Distancia de Edición de Levenshtein:

Distancia De Levenshtein - Ejemplos

	Cadena 1	Cadena 2	Distancia de Levensht	Operaciones
1	gato	pato	1	Sustitución de "g" por "p"
2	silla	salir	2	Sustitución de "i" por "a", Sustitución de "a" por "i"
3	casa	calle	3	Sustitución de "s" por "l", Inserción de "l", Sustitución de "a" por "e"
4	camino	camion	1	Sustitución de "o" por "n"

Recuperación de Textos (RTT)

Recuperación de Textos

- **Consulta de Términos Parecidos:**

```
int LevenshteinDistance(char str1[1..lenStr1], char str2[1..lenStr2])
   // d is a table with lenStr1+1 rows and lenStr2+1 columns
   declare int d[0..lenStr1, 0..lenStr2]
   // i and j are used to iterate over str1 and str2
   declare int i, j, cost

   for i from 0 to lenStr1
       d[i, 0] := i
   for j from 0 to lenStr2
       d[0, j] := j

   for i from 1 to lenStr1
       for j from 1 to lenStr2
           if str1[i] = str2[j] then cost := 0
                                else cost := 1
           d[i, j] := minimum(
                                d[i-1, j] + 1,     // deletion
                                d[i, j-1] + 1,     // insertion
                                d[i-1, j-1] + cost   // substitution
                            )

   return d[lenStr1, lenStr2]
```

Motores de Búsqueda

Motores de Búsqueda

Como es el Motor de Búsqueda de Google?

Google utiliza una combinación de varios modelos de recuperación de información y algoritmos avanzados, que evolucionan constantemente. A continuación se destacan algunos elementos clave que forman parte de su infraestructura de búsqueda

Recuperación de Textos (RTT)

Google

1. Modelo Vectorial (TF-IDF)

Modelo vectorial basado en TF-IDF (Frecuencia de Término - Frecuencia Inversa de Documentos).

Este modelo asigna un peso a cada término en función de su importancia dentro de un documento y en el conjunto total de documentos.

TF (Frecuencia del Término): Mide cuántas veces aparece un término en un documento.

IDF (Frecuencia Inversa de Documentos): Evalúa cuán común o raro es un término en todos los documentos, dándole menos peso a términos muy frecuentes y más a los poco comunes.

Google utiliza una versión avanzada de este enfoque.

Google

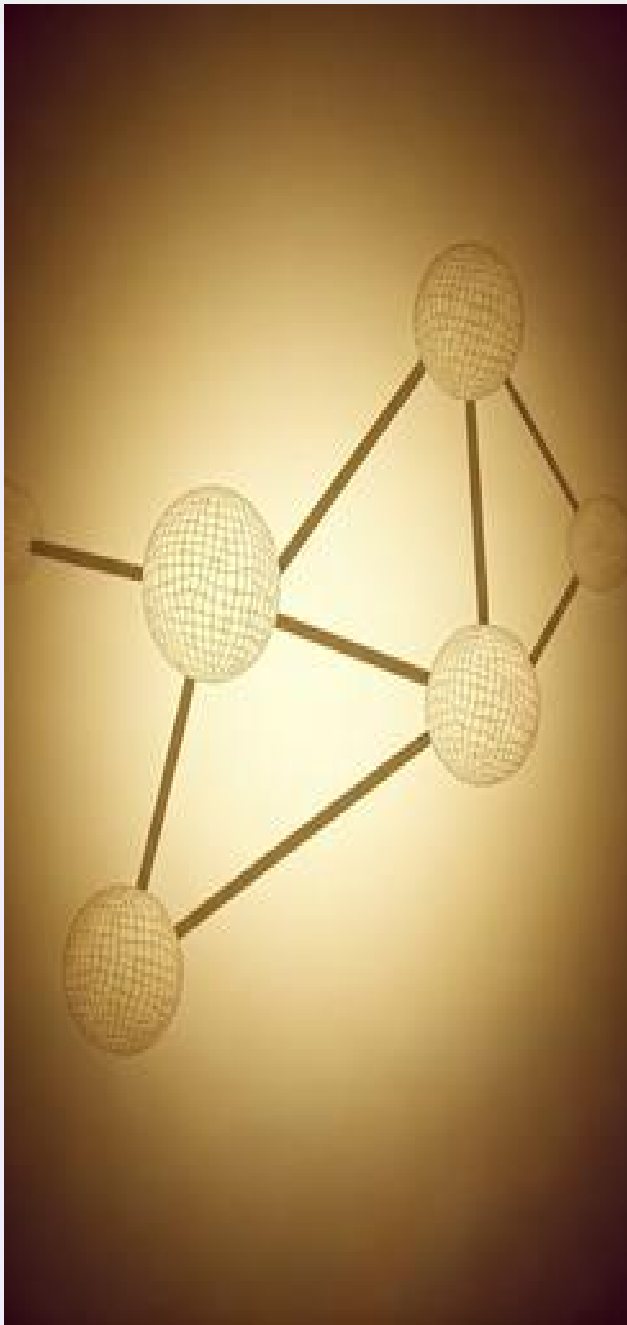
2. PageRank y Redes de Enlaces

Uno de los algoritmos más conocidos que Google desarrolló en sus primeros días es **PageRank**, que valora los documentos en función de la cantidad y la calidad de los enlaces que apuntan hacia ellos.

- **PageRank** mide la importancia de una página en función de cuántas otras páginas la enlazan, y cuán importantes son esas páginas que proporcionan enlaces.

Las páginas con más enlaces relevantes reciben una mayor clasificación en los resultados de búsqueda.

Este método permitió a Google superar a otros motores de búsqueda iniciales, que simplemente basaban su relevancia en los términos de búsqueda sin considerar la calidad del contenido enlazado.



Recuperación de Textos (RTT)

Google

3. Modelos de Aprendizaje Automático (Machine Learning)

En la actualidad, Google utiliza aprendizaje automático para mejorar la relevancia de los resultados de búsqueda. Esto incluye modelos avanzados como:

- **RankBrain:** Un sistema de inteligencia artificial lanzado en 2015, que permite a Google interpretar consultas complejas y ambiguas. RankBrain aprende de los datos pasados y puede ajustar el ranking de los resultados en función de cómo los usuarios interactúan con ellos (por ejemplo, si hacen clic en un resultado y luego regresan rápidamente a la búsqueda, el sistema puede aprender que el resultado no fue relevante).

- **BERT** (Bidirectional Encoder Representations from Transformers): Un modelo basado en redes neuronales profundas lanzado en 2019, que permite a Google entender mejor el contexto de las palabras en las consultas de los usuarios. BERT se utiliza para entender la semántica del lenguaje de forma más precisa, especialmente en consultas conversacionales o frases complejas.

Recuperación de Textos (RTT)

Google

4. Factores Contextuales y Personalización

Google también utiliza muchos factores contextuales que van más allá del texto de la consulta y los documentos. Algunos de estos incluyen:

- **Localización:** Los resultados se personalizan según la ubicación del usuario.
- **Historial de Búsquedas:** Las preferencias del usuario (basadas en búsquedas anteriores) pueden influir en los resultados.
- **Dispositivo:** Google ajusta los resultados según el dispositivo que se está utilizando (por ejemplo, móvil o escritorio).

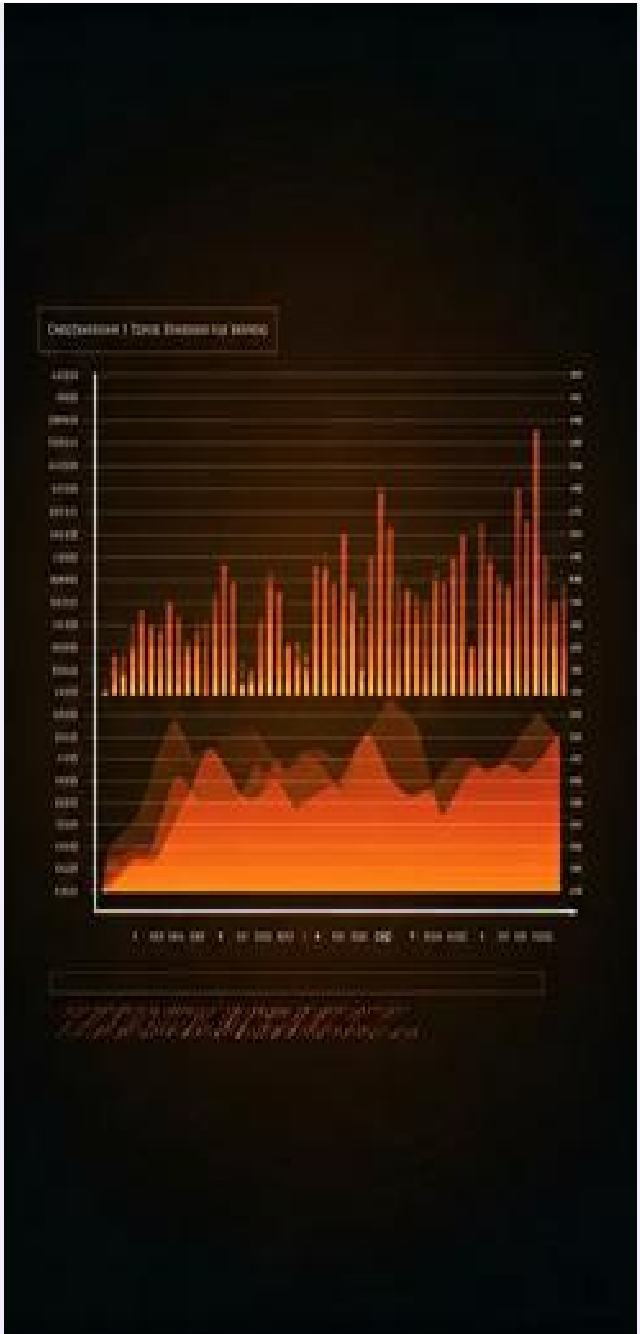
Recuperación de Textos (RTT)

Google

5. Modelos Basados en Redes Neuronales y Semántica (NLP)

Google también utiliza modelos basados en redes neuronales para mejorar la comprensión de la semántica y el significado detrás de las consultas de los usuarios.

- **Procesamiento del Lenguaje Natural (NLP)** permite que Google entienda mejor las consultas con sinónimos, frases largas o preguntas complejas.



Recuperación de Textos (RTT)

Google

6. Factores de Calidad y Experiencia del Usuario

Además de los factores algorítmicos clásicos como TF-IDF, PageRank y BERT, Google tiene en cuenta cientos de señales relacionadas con la calidad del contenido:

- **Velocidad de la página:** Las páginas más rápidas se clasifican mejor.
- **Seguridad (HTTPS):** Las páginas con HTTPS tienen prioridad sobre las no seguras.
- **Compatibilidad con móviles:** Las páginas optimizadas para dispositivos móviles son mejor valoradas, especialmente con la iniciativa mobile-first.

Recuperación de Textos (RTT)

Google

6. Conclusión

- **TF-IDF y modelos vectoriales** para ponderar la relevancia de los términos.
- **PageRank** para medir la calidad y autoridad de las páginas mediante enlaces.
- **Modelos de aprendizaje automático** como RankBrain y BERT para interpretar mejor las consultas de los usuarios y ofrecer resultados más precisos.
- **Factores contextuales y señales de experiencia del usuario** para personalizar los resultados y mejorar la calidad.

Este enfoque multinivel y basado en aprendizaje automático es lo que le permite a Google ofrecer resultados altamente precisos y relevantes, adaptados a cada usuario y consulta.